

SEM7 Self-Healing CI/CD

An ML-assisted DevOps pipeline that predicts risky code changes, catches failing tests, proposes a fix, and repairs the code only after a human approves - then proves the fix with real tests.

[Agentic AI course](#)[IDT / DevOps course](#)[Python](#) · [FastAPI](#) · [Streamlit](#) · [PostgreSQL](#) · [Docker](#) · [GitHub Actions](#)

Status as of	2 October 2026
Repository	github.com/Lakshay0916/SEM7 (private)
Code described here	branch <code>feature/devops-self-healing</code> (commit <code>ac0375e</code>); <code>main</code> holds the earlier version
Automated tests	84 passing locally (+5 PostgreSQL-only), 81 passing inside Docker on PostgreSQL
CI status	All 4 GitHub Actions jobs green

Contents

1. The project in one minute
2. The big picture
3. What happens when code changes (steps 1-5)
4. The ML risk model
5. The self-healing loop (step 6)
6. Safety rules
7. **The IDT / DevOps part**
8. Proof that it works
9. How to run it yourself
10. What comes next
11. Glossary

1. The project in one minute

Software teams constantly push code changes. Sometimes a change breaks something, the automated tests fail, and a developer has to stop, read the error logs, find the cause, fix it, and test again. This project automates as much of that as is safe - while keeping a human in control.

In simple words: think of it as a careful assistant for your code. It looks at every change, warns you if it looks risky, runs the tests for real, and if something breaks it explains why and suggests a fix. It only touches the code after you say "yes", it never touches the main version, and it proves its fix by running the tests again.

The same codebase serves two courses:

Course	Focus	Status
IDT / DevOps	Quality gate, real CI tests, a human-approved self-healing loop, Docker (3 containers), GitHub Actions, reports	Implemented and tested
Agentic AI	AI agents (using a large language model) and RAG (searching project knowledge) to investigate failures and plan smarter fixes	Next phase - the code already has the slot they plug into

One phrase sums up how the pieces divide the work: **"ML predicts, RAG retrieves, LLM reasons, agents act, and tests verify."** Today, the "reasoning" part uses simple, transparent rules instead of an AI model. Everything else around it is real.

2. The big picture

Every change travels through the same path. The top row runs on every change. If the tests fail, the lower rows - the self-healing loop - take over.

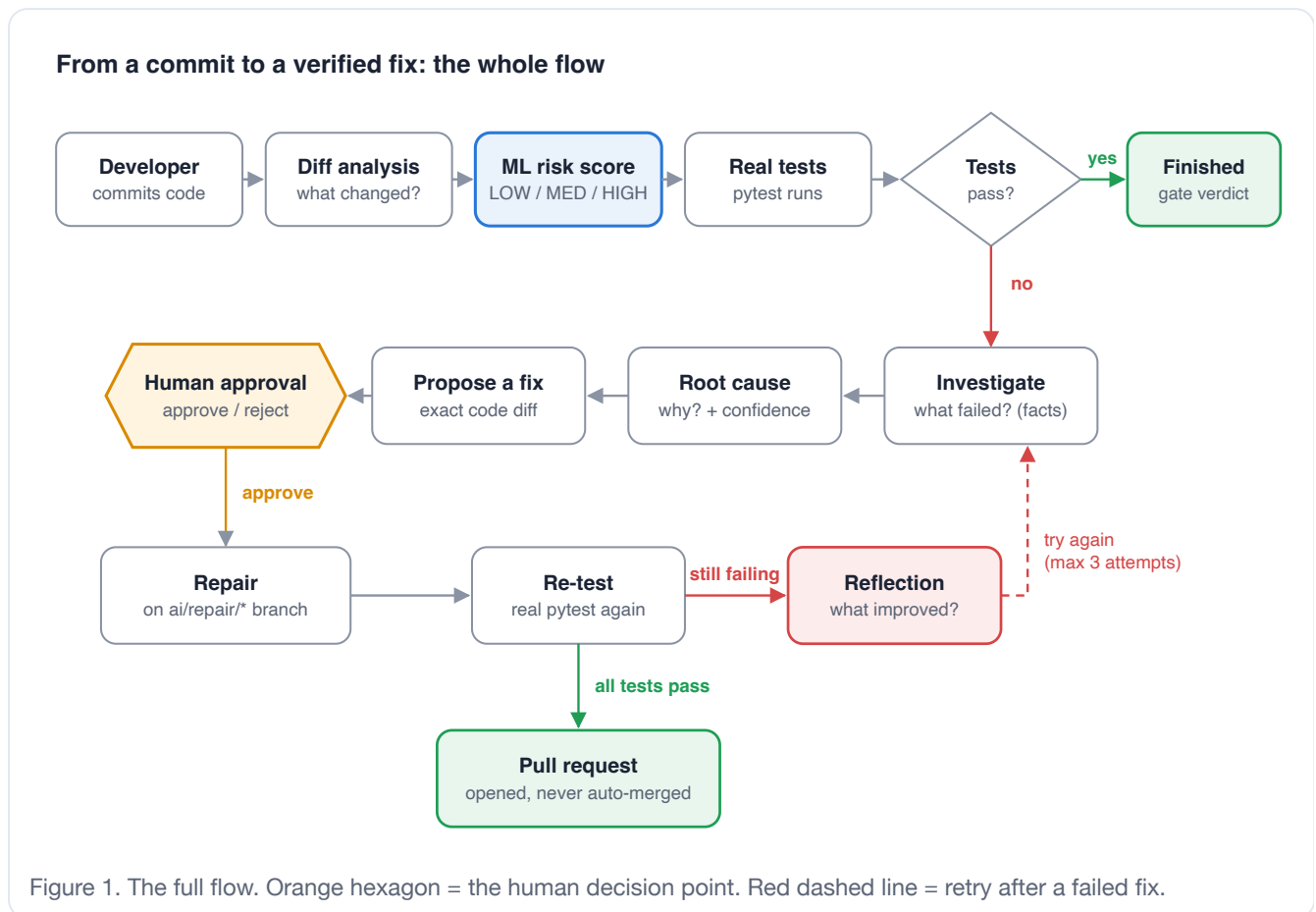


Figure 1. The full flow. Orange hexagon = the human decision point. Red dashed line = retry after a failed fix.

Key idea: the system can investigate and suggest on its own, but changing code requires a person's approval, and "fixed" is decided only by real test results - never by the system's own opinion.

3. What happens when code changes (steps 1-5)

To demonstrate this safely, the project uses a small **calculator** program (add, subtract, multiply, divide, average) with 9 tests. Each **scenario** deliberately breaks it in a known way. Example used below -

Scenario 1: someone changes `add(a, b)` to return `a - b`.

1 Create a safe copy and the developer's commit

What happens: The calculator is copied into a brand-new, separate git repository. The healthy code is saved on the `main` branch. Then the scenario's change is saved (committed) on a separate **feature branch**, exactly like a developer would do.

Why it matters: Nothing real is ever at risk, every run starts from the same point, and `main` always stays healthy as a reference.

Scenario 1 result: Feature branch `feature/refactor-add`, commit message "Refactor add() helper".

2 Work out exactly what changed

What happens: The feature branch is compared with `main` (a **diff**). Each changed file is classified (source code, test, settings, dependency list), and the changed lines are mapped to the exact **functions** they belong to.

Why it matters: The next steps need to know what changed. A failing `test_add` right after `add()` changed is strong evidence.

Scenario 1 result: 1 file changed, 1 line added, 1 removed, function changed: `add`.

3 Predict how risky the change is (ML)

What happens: The change is turned into 27 numbers (how big it is, how many files, were tests touched, etc.). A machine-learning model trained on 3,625 real commits from 9 open-source projects gives a risk score from 0 to 1 and a level: LOW, MEDIUM or HIGH - plus the reasons.

Why it matters: An early warning before tests run, useful for deciding what to review carefully. It is a hint, not proof.

Scenario 1 result: Risk HIGH (score 0.578). Reason: "source changed without accompanying test changes".

4 Run the real tests

What happens: `pytest` (the Python testing tool) is actually executed, with a time limit. The result is read from `pytest`'s own machine-readable report. The verdict is PASS only if `pytest` finished normally, at least one test ran, and nothing failed.

Why it matters: Tests are the ground truth. A timeout, crash or "no tests found" can never be reported as a pass.

Scenario 1 result: FAIL: 7 passed, 2 failed. `test_add`: expected 5 but got -1.

5 Package the evidence

What happens: If tests failed, everything known is bundled into one structured record (a **FailureEvent**): the commit, branch, failing tests, error messages, logs and the diff. Every step is also saved in the database with a timestamp.

Why it matters: The next stage works only from these facts, so it cannot invent logs or results - and anyone can audit the history later.

Scenario 1 result: 2 failing tests recorded, workflow state = FAILED.

The three demo scenarios and what they show:

Scenario	What is broken	Risk	Tests	Shows
1 - Simple bug	<code>add()</code> subtracts instead of adding	HIGH (0.578)	FAIL 7/9	One small fix repairs it
2 - Higher-risk change	Nothing - a broad change (4 files, new dependency, security module)	MEDIUM (0.314)	PASS 9/9	Risk is not the same as a bug
3 - Failed first repair	<code>divide()</code> uses floor division AND lost its zero check	HIGH (0.443)	FAIL 6/9	First fix is not enough; the system retries

4. The ML risk model

The model answers one question: **"How likely is this kind of change to break the tests?"** It does not find the bug.

Where the training data came from

There was no history for the demo project, so real history was collected (**mined**) from 9 well-known open-source Python projects on GitHub: Flask, Click, httpx, Rich, Requests, Alembic, attrs, pip-tools and Poetry. For each commit, the project's real automated test result (GitHub Actions) was downloaded and the commit's changes were measured.

What was collected	Result
Commits with a clear test result	3,625
Commits whose tests really failed	323 (8.9%)
Projects	9 (largest: Click 934 commits, Poetry 861; smallest: Rich 83)

Cleaning the labels mattered. Flask runs one test job against unreleased versions of its dependencies; that job fails no matter what the commit does. Counting it made Flask look 69% broken; ignoring it gives the true 23.5%. Runs that failed before any step even started (server problems, not code problems) were also dropped.

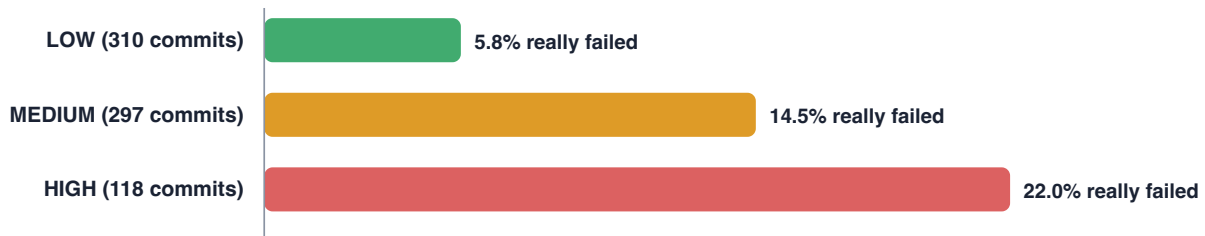
How it was trained fairly

- **Time-ordered split:** for each project, the oldest 70% of commits trained the model, the next 10% chose settings, and the newest 20% tested it - like predicting the future from the past.
- **No peeking:** "history" features (e.g. "did the previous commit fail?") only use results that were already known at that moment. An automated test checks this.
- **Compared with simple rules:** the model only counts as useful if it beats rules like "always predict pass" or "the previous commit failed".

Real results (on the newest 20% of commits, never seen in training)

Model	ROC-AUC	PR-AUC	Plain meaning
Full model (change + CI history)	0.663	0.232	Clearly better than chance; beats the best simple rule
Diff-only model (used for the demo)	0.604	0.161	Weaker - a change alone says only a little
Simple rule: "previous commit failed"	0.572	0.170	The bar the model must beat
Always guess "pass"	0.500	0.120	Pure chance

Does a higher risk level mean more failures? Yes



Held-out test commits the model never saw. The higher the predicted level, the more often CI really failed.

Figure 2. Full model on held-out commits: LOW 5.8%, MEDIUM 14.5%, HIGH 22.0% actually failed.

Honest takeaway: the risk model gives a real but modest signal. That is why the system treats risk as a reason for extra caution, never as a replacement for running the tests (see the quality gate in section 7).

5. The self-healing loop (step 6)

When tests fail, the system tries to repair the code - carefully, in small steps, with a person approving every change.

Stage	What it does (Scenario 1 example)
Investigate	Lists the facts: which tests failed and with what error; which functions the commit changed. Guesses are labelled as <i>hypotheses</i> , never mixed with facts. "test_add failed: expected 5, got -1" · "commit changed add()"
Root cause	Explains why, with a confidence level. "The commit changed the operator in add() from + to -" (80% confidence)
Propose a fix	Shows the exact change as a red/green diff, the tests to run, the risks of the fix and how to undo it. Nothing has been changed yet.
Human approval	You click Approve or Reject. Rejecting changes nothing; you can ask it to re-analyse or stop.
Repair	Applies only the approved change, on a new branch <code>ai/repair/<id></code> - never on <code>main</code> or the developer's branch.
Re-test	Runs the full test suite again for real. 9 of 9 pass.
Reflection	If tests still fail: records what improved, what is still broken and whether anything new broke, then proposes a better fix (which needs approval again). Maximum 3 attempts.
Pull request	When tests pass, a pull request description is written from the recorded evidence: problem, cause, fix, tests, approvals. It is never merged automatically.

How the fixes are chosen today (rule-based, no AI)

The current strategy is deliberately simple and explainable. It always tries the **smallest** fix first:

- Restore the operator:** if a changed line differs from the original only by an operator (+ to - , / to //), put the original operator back.
- Restore the function:** if tests still fail, restore the whole affected function to its healthy version from `main`.
- If neither applies, it stops and hands over to a human.

Scenario 3: two attempts, measured by real tests



Each bar is a real pytest run. Fix 1 was only partly right; reflection noticed the one remaining failure (divide by zero) and the system proposed a broader fix 2.

Figure 3. Scenario 3 measured by real test runs: 6/9 -> 8/9 after fix 1 -> 9/9 after fix 2, then a pull request.

Built to grow: AI agents later plug into the same slot

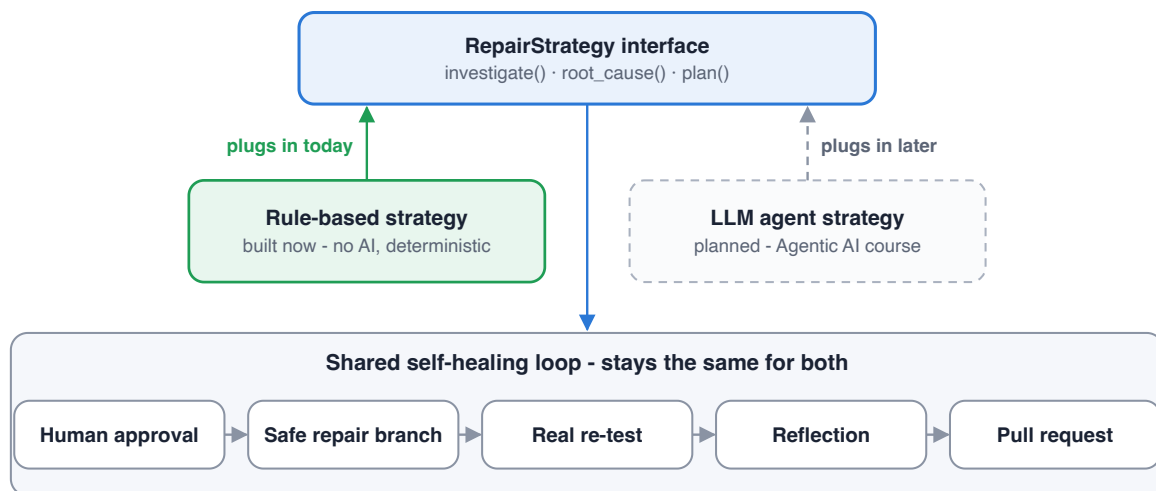


Figure 4. The fixing 'brain' is a replaceable part. Today it is rule-based; the Agentic AI course will plug AI agents into the same slot.

6. Safety rules

These are enforced in code and checked by automated tests - not just promised.

The system never...	How it is enforced
changes <code>main</code>	When the AI side uses git, any commit, reset or push to <code>main / master</code> is refused. Its branches must start with <code>ai/repair/</code> . There is a live "try it" button in the dashboard.
changes code without approval	A state machine only allows the step "approved -> repairing". Skipping approval is impossible.
edits the tests to make them pass	Repairs may not touch test files, settings or dependency files.
makes an unclear edit	The text to replace must appear exactly once in the file.
claims a pass without proof	PASS only comes from a real pytest run; timeouts and crashes are reported as such.
retries forever	Maximum 3 repair attempts, then it stops and asks a human.
merges a pull request	Pull requests are only opened, never merged.
leaks secrets	Passwords/tokens come only from environment settings and are hidden in logs and the database.

7. The IDT / DevOps part

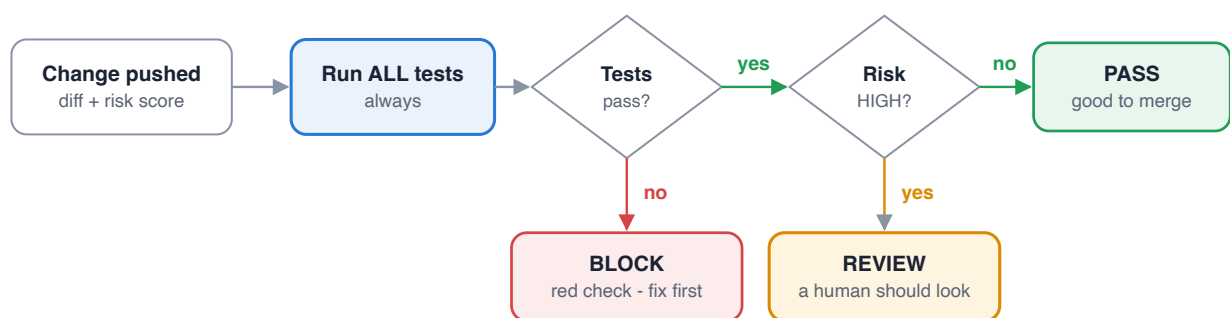
DevOps is about getting code from a developer to users quickly and safely: automated testing, consistent environments, automatic checks on every change, and clear records. This section explains everything built for the IDT course.

Before the IDT work	After the IDT work
Pipeline stopped at "tests failed"	Self-healing loop: investigate -> approve -> repair -> re-test -> pull request
Risk score shown, but not used	Risk-aware quality gate : PASS / REVIEW / BLOCK
Ran only on one laptop	Docker : 3 containers that run the same on any computer
Everything in one program	Separate UI , API and PostgreSQL database
Tests run by hand	GitHub Actions : 4 automatic checks on every push
Only tested on macOS	Also verified on Windows and inside Linux containers
Results only on screen	Exportable reports and pull-request descriptions

7.1 The quality gate

A **quality gate** is an automatic checkpoint that decides whether a change is good enough to continue. Ours combines the real test result with the ML risk score:

The quality gate: a traffic light for every change



Risk can only ADD caution (REVIEW). It never skips tests and never turns a failing build green.

Figure 5. The quality gate. Tests always run first; risk can only add a review flag.

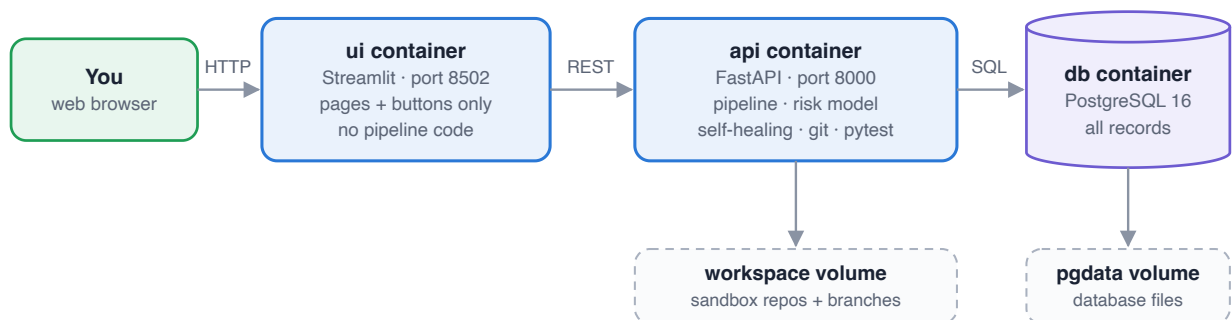
On GitHub this shows up as a green tick (PASS), a yellow warning (REVIEW) or a red cross (BLOCK), and a full report appears in the run summary. Example from the real push of this branch: **PASS** - 75 tests passed, risk MEDIUM (0.317).

Why not skip tests for low-risk changes? The original plan considered sending HIGH-risk changes straight to review without testing. We chose not to: the demo's risk model is only modest (ROC-AUC 0.60), so skipping tests based on it would let real bugs through.

7.2 Three containers: UI + API + PostgreSQL

A **container** is a sealed box that holds a program together with everything it needs to run, so it behaves the same on every computer. The project is split into three containers, each with one job:

Three containers, each with one job



Start-up order is enforced by health checks: db healthy -> api healthy -> ui starts.
The database is only reachable inside the Docker network - not from your computer.

Figure 6. The three containers and how they talk to each other.

Container	Technology	Its job
ui	Streamlit (Python web UI)	Shows the dashboard and sends your button clicks to the API. It contains no pipeline code - which proves the front end and back end are truly separate. Image size: 824 MB.
api	FastAPI (Python web API)	Does all the work: runs pipelines, the risk model and the self-healing loop, with git and pytest inside. Has interactive documentation at <code>/docs</code> . Image size: 1.01 GB.
db	PostgreSQL 16	Stores every workflow, event and artifact permanently. Not reachable from outside the Docker network.

7.3 How Docker Compose runs them

Docker Compose starts all three containers with one command and wires them together. Important details:

- **Health checks and start-up order:** each container reports whether it is healthy. The API waits for a healthy database; the UI waits for a healthy API.
- **Volumes (permanent storage):** `pgdata` keeps the database and `workspace` keeps the sandbox repositories, so data survives restarts.
- **Configuration from a `.env` file:** database user and password, and the ports - so nothing secret is hard-coded.

- **Configurable ports:** if 8000 or 8502 is busy, run `API_PORT=8010 UI_PORT=8512 docker compose up`.
- **Helper services:** `tests` runs the test suite inside the API image against PostgreSQL; `cli` runs command-line scripts.

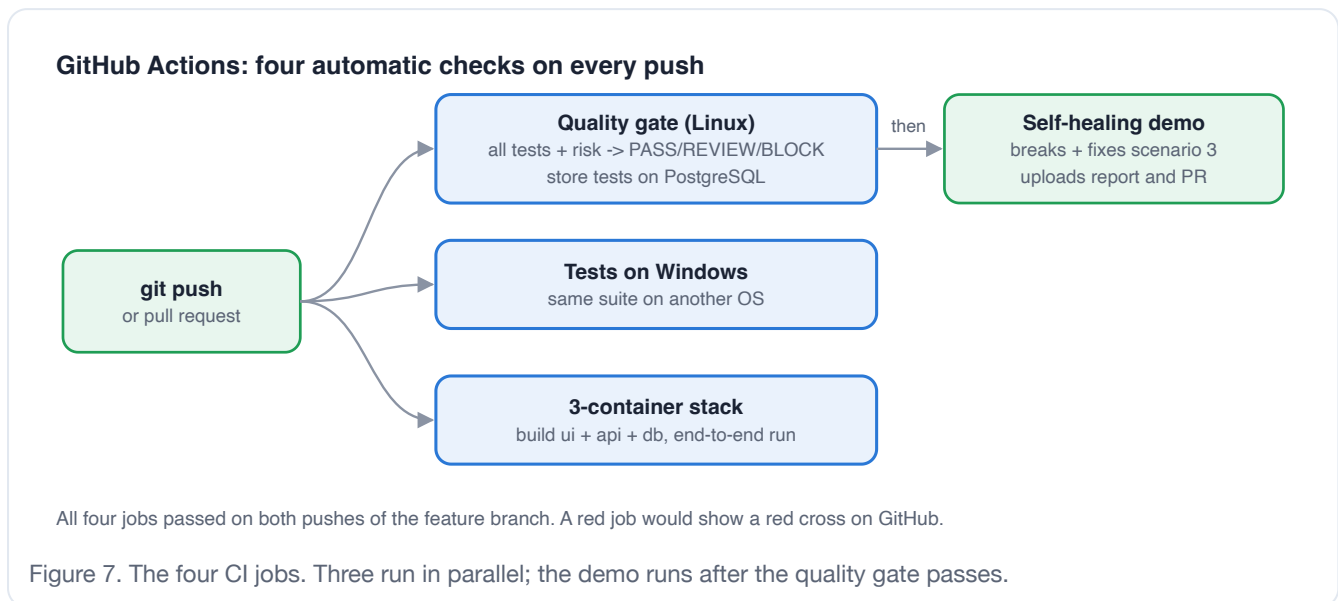
7.4 The API (the backend's front door)

Endpoint	What it does
<code>GET /health</code>	Is the API up, which database is used, which risk model is loaded
<code>GET /scenarios</code>	List the demo scenarios
<code>POST /workflows</code>	Run a scenario: commit, diff, risk, tests, and propose a fix if it fails
<code>GET /workflows</code> , <code>GET /workflows/{id}</code>	List runs / get the full history of one run
<code>POST /workflows/{id}/approve</code>	Approve the proposed fix: repair, re-test, then pull request or reflection
<code>POST /workflows/{id}/reject</code> , <code>/abort</code> , <code>/propose</code>	Reject, stop, or re-analyse
<code>POST /workflows/{id}/safety-check</code>	Try an AI commit to main - must be blocked
<code>GET /risk/model</code> , <code>GET /risk/evaluation</code>	Risk model details and its evaluation results

Trying to approve when nothing is waiting for approval returns an error (HTTP 409) - the "no repair without approval" rule holds over the API too.

7.5 GitHub Actions (automatic checks on every push)

GitHub Actions is GitHub's built-in automation: every time code is pushed, GitHub's own servers run the checks below and show the result next to the commit.



Job	What it proves	Result
Quality gate (Linux)	All tests pass, the gate gives its verdict, and the database code also works on a real PostgreSQL server	Passed
Tests (Windows)	The project works on Windows too (a teammate uses Windows)	Passed
3-container stack	Builds ui + api + db, starts them in order, runs Scenario 3 end to end through the API, runs tests in the container	Passed
Self-healing demo	Breaks and repairs Scenario 3 automatically; the report and pull request can be downloaded from the run page	Passed

7.6 Working on every operating system

Making the Windows job pass meant fixing three classic cross-platform problems: file paths (Windows uses `\` instead of `/`), text encoding (Windows does not default to UTF-8, which breaks symbols like arrows), and line endings (Windows adds an extra character at the end of each line).

7.7 Reports

- **Workflow report:** any run can be exported as a readable Markdown report - what broke, why, each attempt, who approved, test results, pull request.
- **Pull-request description:** written automatically from recorded evidence only.
- **CI gate summary:** shown on every GitHub Actions run.

7.8 The dashboard

The Streamlit dashboard explains every step on screen ("what happens" and "why it matters"), shows the risk gauge, the real test output, the proposed fix as a diff with **Approve / Reject** buttons, each repair

attempt, reflection notes and the final pull request. It supports light and dark mode and has pages for the dashboard, how it works, running a pipeline, workflow history and the risk model.

8. Proof that it works

Check	Result
Automated tests on macOS	84 passed (+5 that run only with PostgreSQL)
Automated tests inside the API container, on PostgreSQL	81 passed
GitHub Actions - first push of the branch	4 of 4 jobs passed
GitHub Actions - after the 3-container change	4 of 4 jobs passed
End-to-end check through the API (Docker)	Scenario 3: FAIL -> reflect -> PASS -> pull request; AI write to main blocked
Browser test of the UI container	Scenario 1 run and approved; repaired and re-tested 9/9
Data stored in PostgreSQL for one Scenario 3 run	27 events and 18 artifacts

What the tests cover, in plain terms:

- **Safety:** AI cannot touch main; no repair without approval; tests and settings cannot be edited; retries stop at 3; rejections change nothing.
- **Honest results:** timeouts, crashes and empty test runs are never reported as passes.
- **ML fairness:** history features never use information from the future.
- **The whole loop:** Scenario 1 fixed in one attempt; Scenario 3 needs two attempts with reflection in between.
- **The API and UI:** every endpoint, error cases, and the UI showing a clear message when the API is down.

9. How to run it yourself

Option A - Docker (recommended, no Python setup)

```
docker compose up --build
# UI: http://localhost:8502      API docs: http://localhost:8000/docs
# ports busy? API_PORT=8010 UI_PORT=8512 docker compose up --build
```

Then in the UI: **Run Pipeline** -> choose **Scenario 3** -> **Run the pipeline** -> scroll to step 6 -> **Approve & apply** (twice).

```

docker compose --profile tools run --rm tests          # run the tests inside Docker
python scripts/e2e_api.py --url http://localhost:8000 # end-to-end check of the
running stack
docker compose down                                   # stop (add -v to delete the
data)

```

Option B - Without Docker (two terminals)

```

conda activate SEM7
uvicorn app.api.main:app --port 8000      # terminal 1: the backend
streamlit run ui/app.py                  # terminal 2: the dashboard

```

Option C - Command line only

```

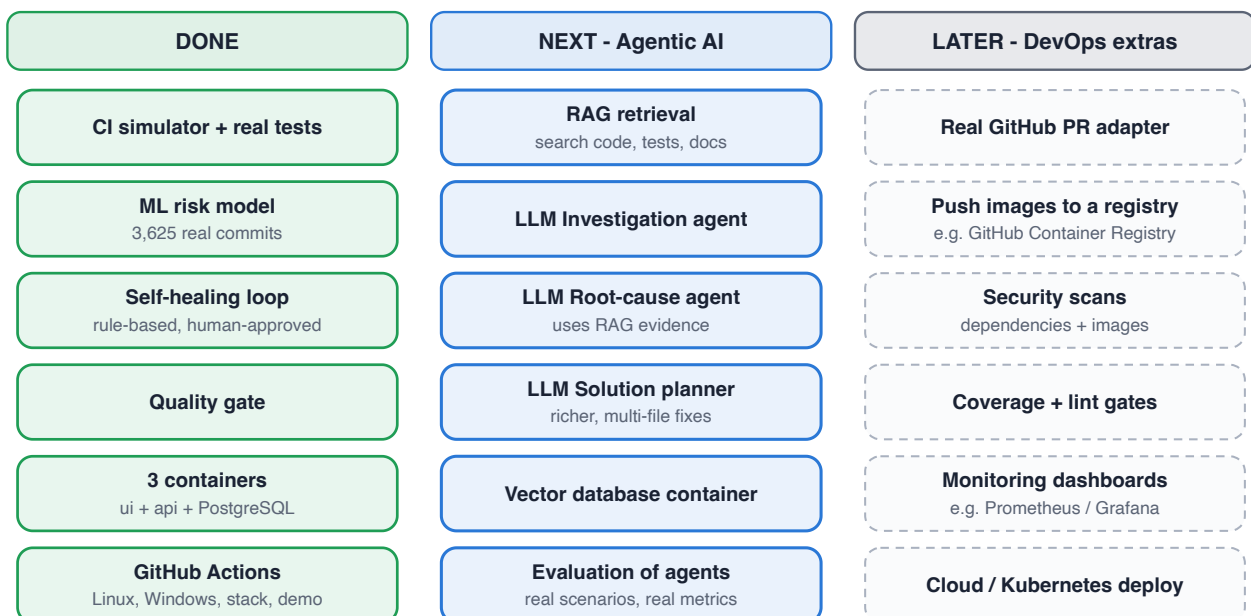
python scripts/run_ci.py --scenario failed_first_repair --heal --approve
python scripts/export_report.py --latest
python scripts/ci_gate.py --base main
pytest

```

10. What comes next

The foundation - pipeline, safety rules, approval, testing, containers and CI - is finished. The next work replaces the simple rule-based "brain" with AI agents, and can optionally extend the DevOps side further.

Roadmap: what is built and what comes next



LATER items are suggestions for extending the IDT work; none of them is built yet.

Figure 8. Roadmap. Green = built, blue = next (Agentic AI), grey dashed = possible DevOps extensions.

Agentic AI (next)

To add	What it will do
RAG retrieval	Search the project's code, tests, documentation and history for the pieces relevant to a failure, and pass them to the agents as evidence.
LLM Investigation agent	Read logs and diffs like a developer would, and summarise what failed - still separating facts from guesses.
LLM Root-cause agent	Explain why the failure happened using the retrieved evidence, with a confidence level.
LLM Solution planner	Propose fixes beyond reverting a change (new logic, several files) - still shown as a diff for approval.
Vector database container	A fourth container to store the searchable knowledge used by RAG.
Agent evaluation	Measure root-cause correctness, fix success rate and attempts needed on real scenarios.

Why this will be easy to add: the three "thinking" steps already sit behind one interface (`investigate` , `root_cause` , `plan`). An AI version only has to fill those three slots. The approval step, safety rules, real re-testing, reflection, pull request, dashboard and database records are reused unchanged.

Possible DevOps extensions (later)

These are suggestions to strengthen the IDT side further; none is built yet:

- **Real GitHub pull requests** instead of the local adapter used for the demo repositories.
- **Container registry:** publish the images automatically (e.g. GitHub Container Registry) after a green build.
- **Security scanning** of dependencies and container images.
- **Coverage and lint gates:** also block changes that reduce test coverage or break code style.
- **Monitoring:** dashboards for pipeline runs, repair success rate and timings (e.g. Prometheus and Grafana).
- **Cloud or Kubernetes deployment** of the three containers.

11. Glossary

Term	Meaning in plain English
Commit	A saved snapshot of code changes, with a message describing them.
Branch	A separate line of work. <code>main</code> is the official version; feature and repair branches hold changes until approved.
Diff	The list of exactly what changed between two versions (removed lines in red, added lines in green).
Pull request (PR)	A request to merge a branch into another, so people can review it first.
CI (Continuous Integration)	Automatically testing every change as soon as it is pushed.
CI/CD	CI plus Continuous Delivery/Deployment - automatically getting tested code out to users.
pytest	The standard tool for running automated tests in Python.
Quality gate	An automatic checkpoint that decides whether a change may continue (PASS / REVIEW / BLOCK here).
Machine learning model	A program that learns patterns from past examples to make predictions about new ones.
ROC-AUC	How well a model ranks risky changes above safe ones. 0.5 = coin flip, 1.0 = perfect.
PR-AUC	A stricter score for rare events like failures. Blind guessing scores the failure rate (about 0.12 here).
Container / Docker	A sealed box with a program and everything it needs, so it runs the same everywhere. Docker builds and runs them.
Docker Compose	A tool that starts several containers together from one file.
Volume	Storage that lives outside a container, so data survives restarts.
Health check	A regular "are you OK?" test a container answers; used to start services in the right order.
API	A set of web addresses a program offers so other programs (like the UI) can ask it to do things.
FastAPI / Streamlit	Python frameworks for building web APIs and web dashboards.
PostgreSQL	A widely used, reliable database server.
GitHub Actions	GitHub's automation that runs jobs (tests, builds) on GitHub's servers after each push.
LLM	Large Language Model - an AI model (like Claude) that understands and writes text and code.

Agent	An AI component given one job (e.g. "find the root cause") and the tools to do it.
RAG	Retrieval-Augmented Generation - first search for relevant information, then give it to the LLM as evidence.
State machine	A fixed list of allowed steps and moves between them (e.g. you can only repair after approval).

All numbers in this guide come from real runs of the code on the branch named on the cover (test runs, GitHub Actions runs and the saved model evaluation).